

数据结构机考学习2：链表

昨天学了顺序表，今天继续学习链表。

因为我之前已经有了一定的代码基础，

所以基础操作部分的代码我只用自己的话来概括一遍，

如果是第一次看的话建议画一下图，

或者在b站看相关的讲解会更加清晰。

基础语法：

结点定义：

```
template<typename T>
struct node {
    T data;
    node<T>* next;
};
```

一个结点有：

数据域，存储结点携带的数据。

指针域，存储下一个结点的地址。

私有成员：

```
template<typename T>
class linklist {
private:
    node<T>* head;
    int length;
```

head 是一个指针。

它可以存储一个 node<T> 类型变量的地址。

基本操作：

1.尾插法创建链表

方法概述：

先new一个头结点。

然后定义tail指针，

接下来要让它一直指向最后一个结点。

循环部分：

new新结点，加入数据并让它的next指向空，

然后让链表中原来最后一个结点的next指向新节点，

所以，新结点就会成为链表中的最后一个结点。

这就是尾插法。

最后再让tail指向新建的结点，即指向最后一个结点，继续循环。

```
linklist(T a[], int n) {  
    head = new node<T>;  
    head->next = NULL;  
    node<T>* tail = head;  
    for (int i = 0; i < n; i++)  
    {  
        node<T>* newnode = new node<T>;  
        newnode->data = a[i];  
        newnode->next = NULL;  
        tail->next = newnode;  
        tail = newnode;  
    }  
    length = n;  
}
```

2.头插法创建链表

方法概述：

先new一个头结点，

接下来要让头结点的next一直指向新建的结点。

循环部分：

new新节点，加入数据，

并且让它的next指向原来链表的头结点，

即head的next,

最后把头结点的next指向新建的结点即可。

注意：

如果是正向遍历a[i]的话，创建的链表的值就会倒过来

```
linklist(T a[], int n, bool flag)
{
    head = new node<T>;
    head->next = NULL;
    for (int i=n-1;i>=0 ;i--)
    {
        node<T>* newnode = new node<T>;
        newnode->data = a[i];
        newnode->next = head->next;
        head->next = newnode;
    }
    length = n;
}
```

3.销毁链表

方法概述：

先定义p指针指向头结点，

while循环：

用q指针指向p保持当前结点，

然后p就指向到下一个结点，

释放当前结点q，

直到p指向NULL为止。

```
~linklist()
{
    node<T>* p = head;
    while (p != NULL)
    {
        node<T>* q = p;
        p = p->next;
        delete q;
    }
}
```

4.按位置和价值查找

比较简单，不再赘述。

```
T get(int i)
{
    node<T>* p = head->next;int j = 1;
    while (p != NULL && j < i)
    {
        p = p->next;j++;
    }return p->data;
}

int locate(T x)
{
    node<T>* p = head->next;int j = 1;
    while (p != NULL && p->data != x)
    {
        p = p->next;j++;
    }
    if (p == NULL) return 0;
    return j;
}
```

5.插入操作

方法概述：

首先用p指针找到第i-1个结点。

```
void insert(int i, T x)
{
    node<T>* p = head;
    int j = 0;
    while (p != NULL && j < i - 1)
    {
        p = p->next;
        j++;
    }
}
```

插入：

先new一个新结点，存数据，

把它的next指向第i-1的结点的next，

最后再把前面结点的指针指向新节点。

```

node<T>* newnode = new node<T>;
newnode->data = x;
newnode->next = p->next;
p->next = newnode;
length++;

```

6.删除操作

同理，这里不再赘述。

```

T remove(int i)
{
    node<T>* p = head;
    int j = 0;
    while (p != NULL && j < i - 1)
    {
        p = p->next; j++;
    }
    node<T>* q = p->next;
    p->next = q->next;
    T value = q->data;
    delete q; length--;
    return value;
}

```

例题：

1.链表逆置

题目：把链表反转，不创建新结点，只修改指针指向。

思路：

其实思路和前插法差不多。

1. 把头结点断开（ head->next = NULL ）
2. 用 p 遍历原链表的每个结点
3. 把每个结点用**头插法**插入到头结点后面

```

void reversed(linklist<T>& list) {
    node<T>* head = list.head;
    node<T>* p = head->next;
    head->next = NULL;
    while (p != NULL) {
        // 头插法：把 p 插到头结点后面
        node<T>* q = p->next;
        p->next = head->next;
        head->next = p;
        p = q; // 继续处理下一个
    }
}

```

2.删除链表中所有值为x的结点

核心思路

用一个**前驱指针** prev 记录当前结点的前一个结点：

- 如果当前结点需要删除： prev->next = p->next ，删除 p ， p 指向下一个
- 如果不需要删除： prev 和 p 都向后移动

```

void deleteall(linklist<T>& list, T x) {
    node<T>* head = list.head;
    node<T>* prev = head;
    node<T>* p = head->next;
    while (p != NULL) {
        if (p->data == x) {
            prev->next = p->next;
            delete p;
            p = prev->next;
            list.length--;
        } else {
            prev = p;
            p = p->next;
        }
    }
}

```

3.找出倒数第k个结点

核心思路

1. 快指针先走 k 步
2. 然后快慢指针一起走
3. 快指针到 NULL 时，慢指针正好在倒数第 k 个

原理：快指针比慢指针多走 k 步，当快指针到终点时，慢指针距离终点还有 k 步，即倒数第 k 个。

```
T findKthFromEnd(linklist<T>& list, int k) {  
    node<T>* head = list.head;  
    node<T>* slow = head->next;  
    node<T>* fast = head->next;  
    // 1. fast 先走 k 步  
    for (int i = 0; i < k; i++) {  
        fast = fast->next;  
    }  
    // 2. slow 和 fast 一起走  
    while (fast != NULL) {  
        slow = slow->next;  
        fast = fast->next;  
    }  
    return slow->data;  
}
```